

2016 年-2017 学年度第一学期 华中科技大学本科生课程考试试卷(A 卷)

(自动化学院 2015 级各专业用)

课程名称: 数据结构 课程类别 ☐ 公共课 ☒ 专业课 考试形式 ☐ 开卷 ☒ 闭卷
所在院系: 自动化学院 专业及班级: 级所有专业 考试日期:
学 号: 姓 名: 任课教师:

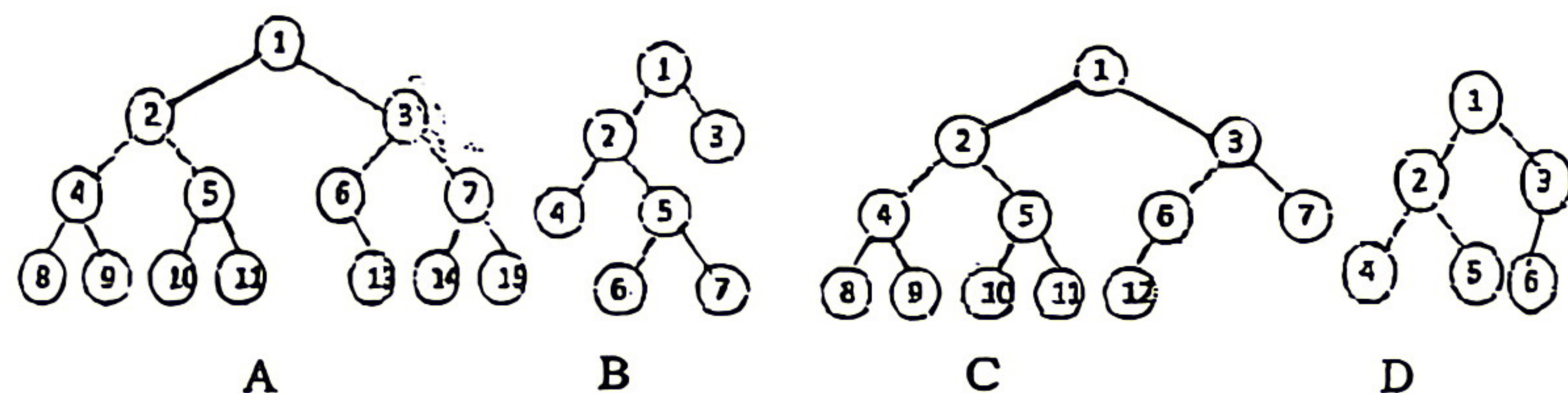
题号	一(26)	二(15)	三(10)	四(10)	五(12)	六(15)	七(12)	总分
得分								

本试题卷面分共 100 分。答题前请写上学号和姓名。

得分	评卷人	一、选择题 (每题两分, 共 26 分)

1. 在有 n 个结点的二叉链表中必定有 C 个空链域, 一棵有 n 个叶子结点的 Huffman 树有 D 个结点。
A $n-1$; B $2n+1$ C $n+1$ D $2n-1$

2. 下面的二叉树中 C 是完全二叉树;



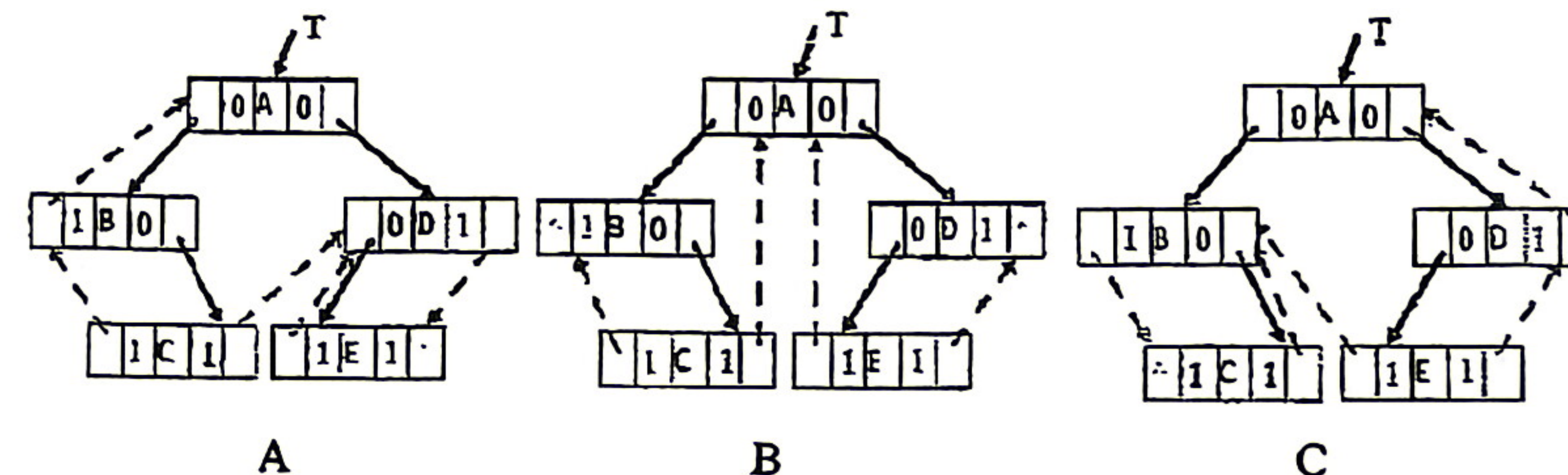
3. 下面三个程序中, C 完成二叉树的后序遍历二叉树;

<pre>void order(TreeNode *bt) { if(bt==NULL) { order(bt->lchild); printf("%d\t", bt->data); order(bt->rchild); } }</pre> A	<pre>void order(TreeNode *bt) { if(bt==NULL){ printf("%d\t", bt->data); order(bt->lchild); order(bt->rchild); } }</pre> B	<pre>void order(TreeNode *bt) { if(bt!=NULL){ order(bt->lchild); order(bt->rchild); printf("%d\t", bt->data); } }</pre> C
--	---	---

4. 图的广度优先搜索算法需要采用 B 实现;

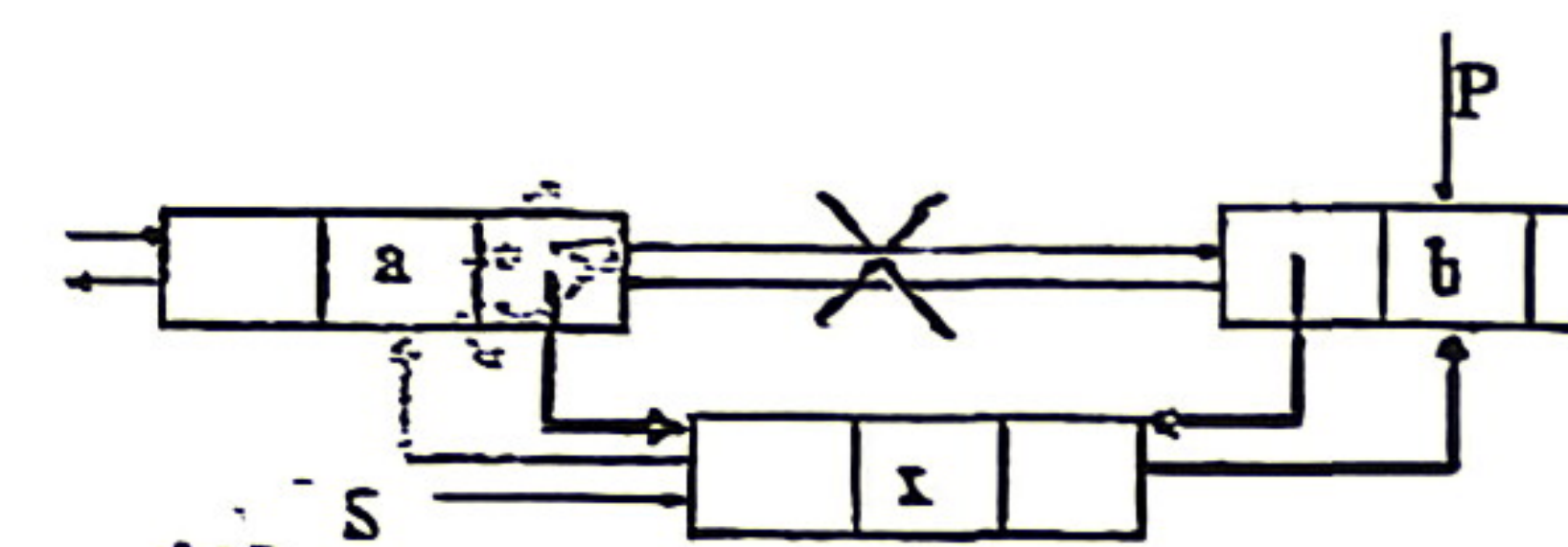
A 线性表; B 队列 C 堆栈 D 广义表

5. 下面的线索二叉树中, B 是中序线索二叉树;



6. 下面的程序和图描述双向链表的插入过程, 在空白处选择合适的语句;

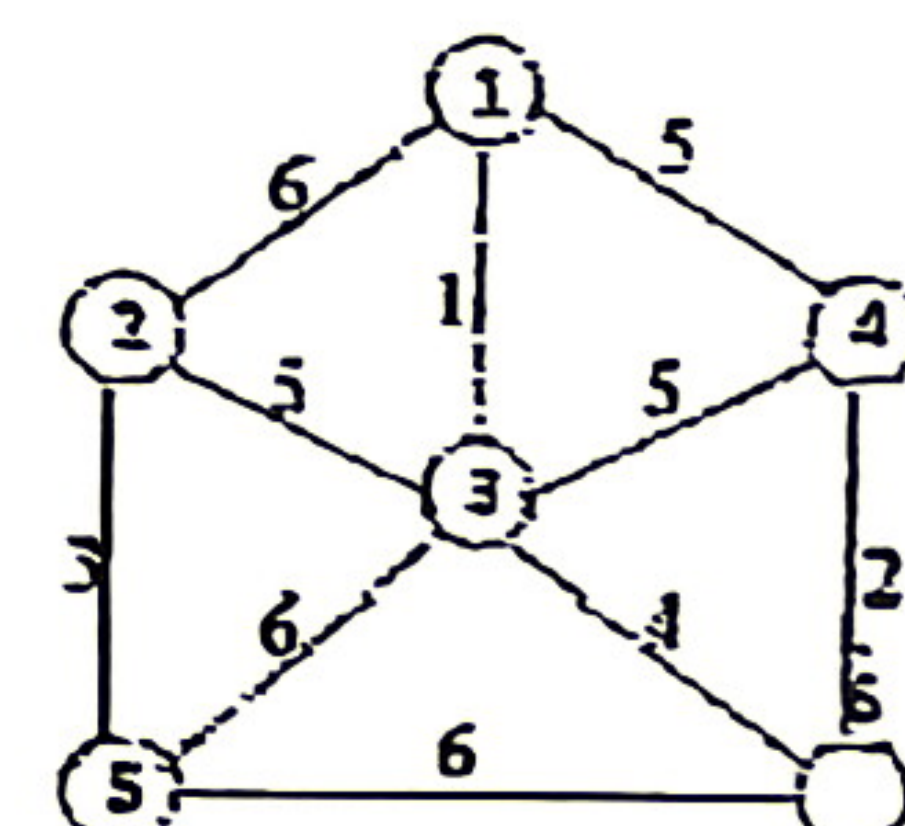
```
Status ListInsert_DuL(DuLinkedList &L, int i, ElemType e) {
    if(!(p = GetElemP_DuL(L, i))) return ERROR;
    if(!(s = (DuLinkedList)malloc(sizeof(DuLNode)))) return ERROR;
    s->data = e;
    s->prior = p->prior; s->next = p;
    p->prior->next = s; p->prior = s;
    return OK;
}
```



- A $p->prior = s->next$ B $p->prior = s$
C $s->next->prior = p$ D $p->prior->next = s$

7. 给定如右图无向图, 求其最小生成树, 在得到最小生成树的过程中, 采用普里姆(Prim)算法得到的第 4 边是 B;

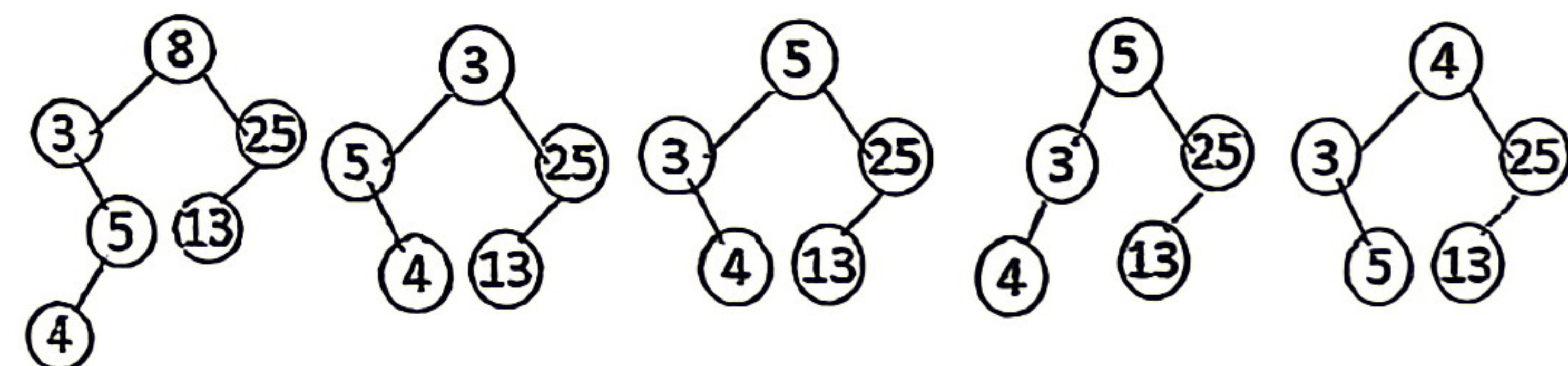
- A (4, 6)
B (2, 3)
C (3, 6)
D (3, 4)



8. 上一题中(第 7 题), 如果把 Prim 算法换成克鲁斯卡尔(Kruskal)算法, 得到的第 4 条边是 C;

- A (4, 6) B (2, 3)
C (3, 6) D (3, 4)

9. 下图左边图 E 为一数字序列对应的二叉排序树, 删掉节点 8 后所得到的二叉排序树为右边 A、B、C 和 D 等 4 个图中 B;

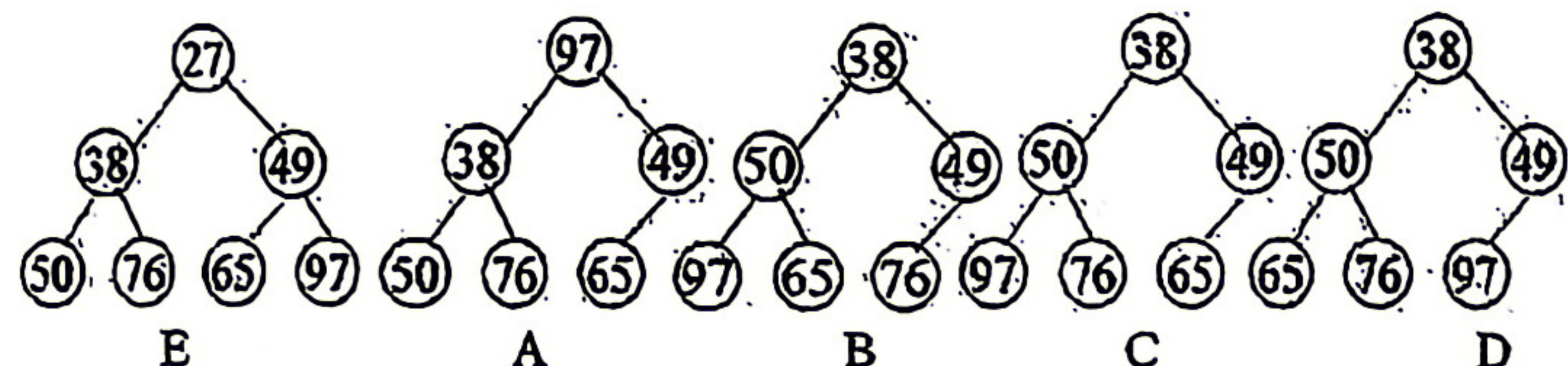


E A B C D

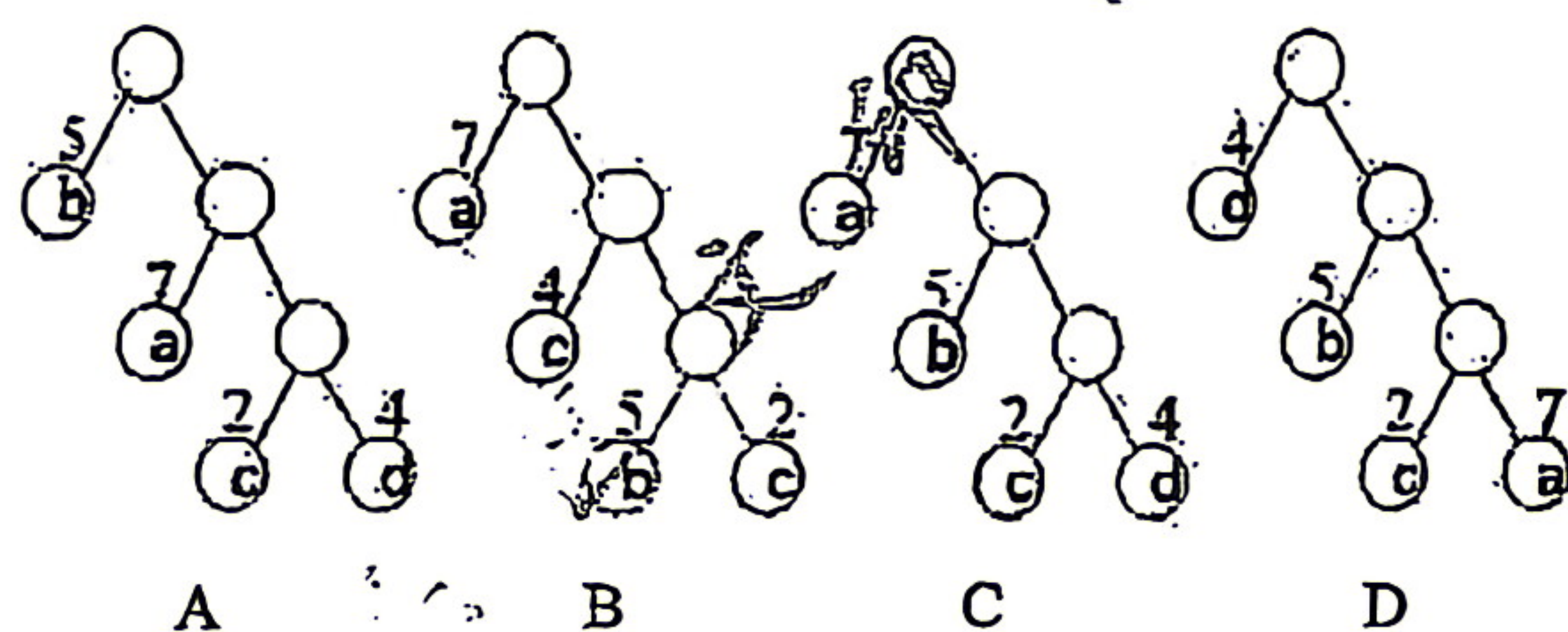
10. 归并排序的时间复杂度是 A，冒泡排序的时间复杂度是 C，堆排序最坏情况下的时间复杂度是 A，快速排序最好情况下的时间复杂度是 A；

A $O(n \log n)$ B $O(\log n)$ C $O(n^2)$ D $O(n)$

11. 下图左边图 E 为根据一数字序列采用堆排序算法所建成的一个堆，该堆排序算法可将数字序列从小到大进行排序，当输出堆顶元素 27 后，调整剩下元素得到的新堆为右边 A、B、C 和 D 等 4 个图中 C；



12. 设 a、b、c、d 四个节点所对应的权值分别为 5, 2, 4, 如下四个二叉树中 C 为对应的 Huffman 树。



13. 如下表长为 11 的哈希表中已填有关键字为 17, 60, 29 的记录，采用开放地址法处理冲突， $H_i = (\text{key} + 2d_i) \text{MOD } 11$ ， $d_i = 1, 2, \dots, 10$ ，现有第 4 个记录，其关键字为 38，应该填在表中 D 处。

0	1	2	3	4	5	6	7	8	9	10
					60	17	29			

A 4 B 5 C 8 D 9

得分	评卷人

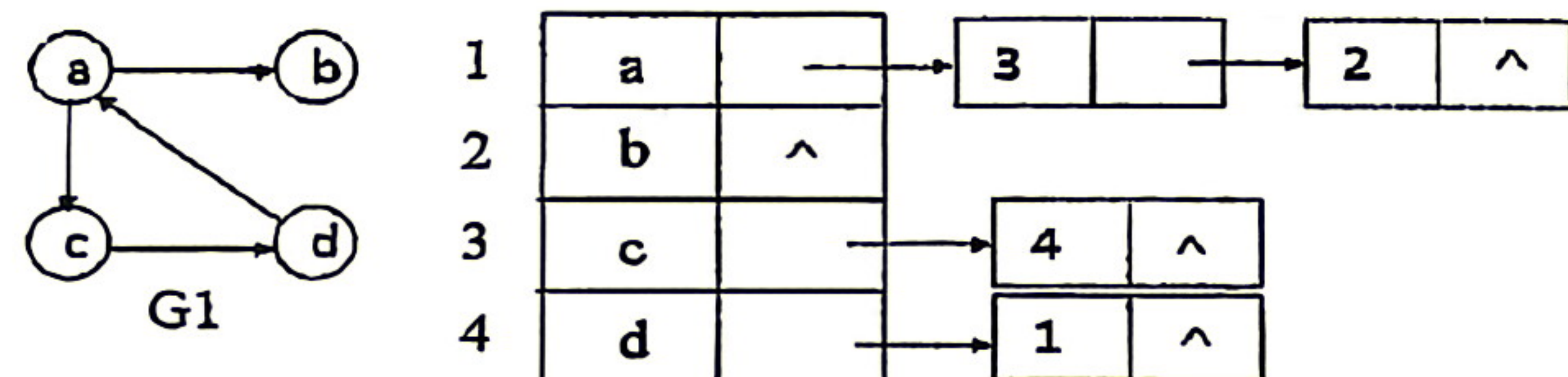
二、简答题 (15)

- 比较分析顺序存储方式和链表存储方式各自优缺点；
- 如何评价一个算法的性能；
- 试分析线性表、堆栈和队列各自的特点，并给出几个数据结构这门课中可以采用堆栈和队列来实现的算法的例子。

得分	评卷人

三、数据结构定义题 (10 分)

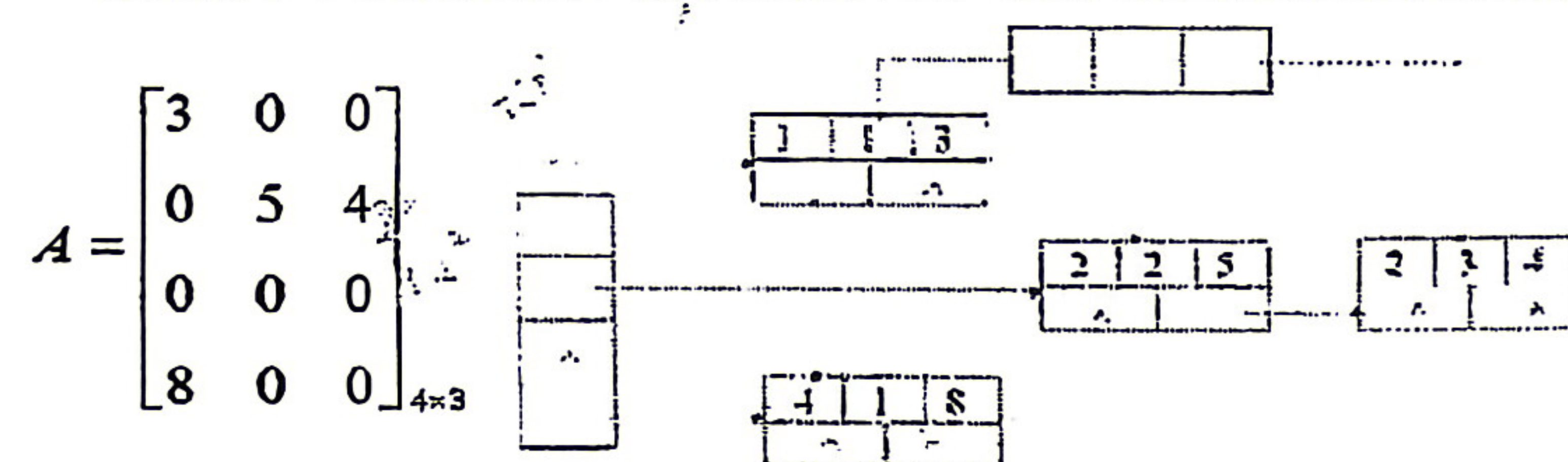
1. 采用如下邻接表的方式来表达图 G1, 写出其完整的数据结构定义:



得分	评卷人

四、给定如下两个单向链表 L_a 和 L_b , 两个链表均按关键字的绝对值从大到小有序, 即是 $|a_n| > |a_{n-1}| > \dots > |a_1|$, $|b_n| > |b_{n-1}| > \dots > |b_1|$, 编程序合并两个链表, 并按从小到大排序, 并分析所编程序的时间复杂度和空间复杂度。(10 分)

2. 采用如下十字链表的方式来表达矩阵 A, 写出其完整的数据结构定义:

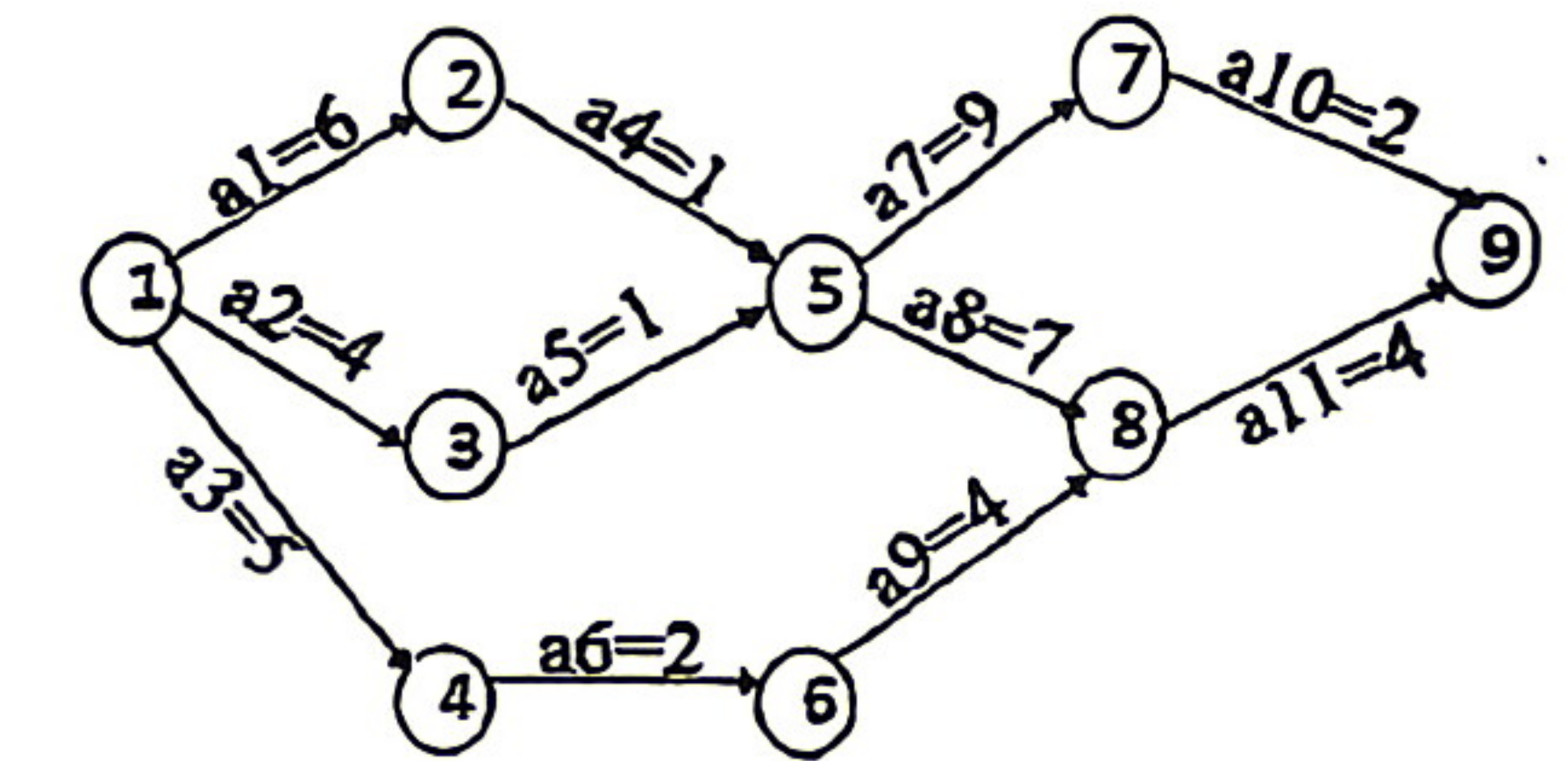


得分	评卷人

五、给定一数字系列 (10, 6, 12, 43, 67, 24, 87, 14, 35, 19)，分别采用希尔排序和快速排序算法对其从小到大进行排序，给出计算步骤，然后对排好序的系列采用折半查找算法查找数据 67，给出计算步骤：(12 分)

得分	评卷人

六、给定如下的有向图：1) 求出其关键路径，给出步骤；2) 采用 Dijkstra 算法求出从节点 1 到其它各节点的最短路径，给出步骤，并指出计算过程中第 4 个得到最短路径的节点是哪一个节点；(15 分)



得分	评卷人

七、程序阅读题，三题中选作两题（12分）

1. 下面的程序为中序遍历二叉树的非递归算法，在空白处填上适当的语句。

```

Status InOrderTraverse(BiTree T, Status (*Visit)(ElemType))
{ // 采用二叉链表存储结构，Visit是对数据元素操作的应用函数。
  stack S;
  BiTree p;
  InitStack(S); Push(S, T);
  while (!StackEmpty(S)){
    while (GetTop(S, p) && p) Push(S, p->lchild);
    Pop(S, p);
    if (!StackEmpty(S)){
      Pop(S, p);
      if (!Visit(p->data)) return ERROR;
      Push(S, p->rchild);
    }
  }
  return OK;
}

```

2. 下面的程序对有向图进行拓扑排序，在空白处填上适当的语句。

```

Status TopologicalSort(ALGraph G) {
  // 有向图G采用邻接表存储结构
  // 若G无回路，则输出G的顶点的一个拓扑序列并返回OK，否则ERROR。
  SqStack S;
  int count, k, i;
  ArcNode *p;
  char indegree[MAX_VERTEX_NUM];
  FindInDegree(G, indegree); // 对各顶点求入度indegree[0..vernum-1]
  InitStack(S);
  for (i=0; i<G.vexnum; ++i)
    if (!indegree[i]) Push(S, i);
  count = 0;
  while (!StackEmpty(S)) {
    Pop(S, i);
    printf(i, G.vertices[i].data); ++count;
    for (p=G.vertices[i].firstarc; p; p=p->nextarc) {
      k = p->adjvex;
      if (--indegree[k]) Push(S, k);
    }
  }
  if (count<G.vexnum) return ERROR; //该有向图有回路
  else return OK;
}

```

```

} // TopologicalSort

```

3. 下面的程序采用三元组表存储矩阵，实现矩阵的快速转置，在空白处填上适当的语句。

```

#define MAXSIZE 12500
typedef struct {
  int i, j;
  ElemType e;
} Triple;
typedef struct {
  Triple data[MAXSIZE+1];
  int mu, nu, tu;
} TSMatrix;
Status FastTransposeSMMatrix(TSMatrix M, TSMatrix &T) {
  T.mu = M.nu; T.nu = M.mu; T.tu = M.tu;
  if (T.tu) {
    for (col=1; col<=M.nu; ++col) num[col]=0;
    for (t=1; t<=M.tu; ++t) ++num[M.data[t].j];
    cpot[1] = 1;
    for (col=2; col<=M.nu; ++col) cpot[col] = cpot[col-1] + num[col-1];
    for (p=1; p<=M.tu; ++p) {
      col = M.data[p].j; q = cpot[col];
      T.data[q].i = M.data[p].j; T.data[q].j = M.data[p].i;
      T.data[q].e = M.data[p].e; ++cpot[col];
    }
  }
  return OK;
}

```